

Ejercicios extra de Conjuntos y JOINS

Bases de Datos

1. Explicación cruzada entre conjuntos y SQL

🔗 Analice la operación de conjuntos All - Odd.

All = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

Odd = {1, 3, 5, 7, 9}

Devuelve los elementos que se encuentra en A que no están B,

En este caso el resultado de All-Odd sería {2,4,6,8,10}

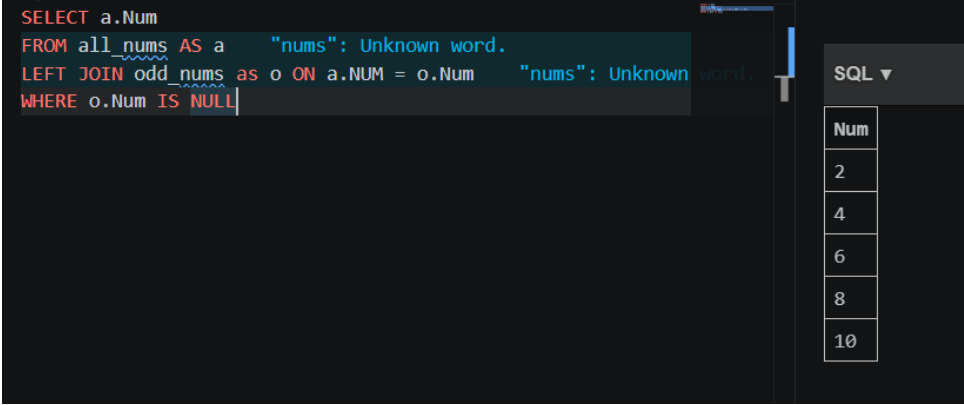
🔗 Explique cómo una operación similar se puede representar en SQL con JOINS.

Se agrega las tablas all_num y odd_num, se le inserta la información correspondiente, solo se utiliza una columna para cada tabla, y se procede a utilizar SELECT para llamar a la tabla, en este caso se le dio apodo "a" para all_num y "o" para ood_num,

¿Qué tipo de JOIN usaría?

Se utiliza LEFT JOIN, esto para que se mantenga toda la información de la tabla de la izquierda y se adjunten coincidencia de la derecha, para ver únicamente los datos de la tabla A que hacen falta en la tabla B se utiliza WHERE o.Num is NULL, esto para que nos muestre únicamente los faltantes y no la tabla completa.

```
SELECT a.Num
FROM all_nums AS a
LEFT JOIN odd_nums as o ON a.NUM = o.Num
WHERE o.Num IS NULL
```



Num
2
4
6
8
10

2. Agrupamiento y conteo cruzado

🔗 Usando las tablas de Books, Customers y Rents:

- Obtenga el número total de veces que cada cliente ha rentado un libro

Para esto llamamos a la columna Name de Customers y contamos los ID´s de Rent, con INNER JOIN buscamos los valores que coinciden entre ambas tablas y con GROUP BY ordenamos que el total se muestre desde Customer ID y Customer name

<pre> SELECT c.Name, COUNT(r.ID) as TotalRents FROM Customers AS c INNER JOIN Rents AS r ON c.ID = r.Customer_ID GROUP BY c.ID, c.Name; </pre>	SQL ▼ <table> <tr> <th>Name</th><th>TotalRents</th></tr> <tr> <td>John Doe</td><td>2</td></tr> <tr> <td>Jane Doe</td><td>3</td></tr> </table>	Name	TotalRents	John Doe	2	Jane Doe	3
Name	TotalRents						
John Doe	2						
Jane Doe	3						

- Ordene de mayor a menor y limite el resultado a los **3 clientes más activos**.

Se agrega ORDER BY Total de renta en Descendente, esto para que vaya de mayor a menor y luego LIMIT 3, para que nos muestre un máximo de 3, en este caso solo hay 2 clientes que rentaron por lo que nos muestra únicamente 2,

<pre> SELECT c.Name, COUNT(r.ID) as TotalRents FROM Customers AS c INNER JOIN Rents AS r ON c.ID = r.Customer_ID GROUP BY c.ID, c.Name ORDER BY TotalRents DESC LIMIT 3; </pre>	SQL ▼ <table> <tr> <th>Name</th><th>TotalRents</th></tr> <tr> <td>Jane Doe</td><td>3</td></tr> <tr> <td>John Doe</td><td>2</td></tr> </table>	Name	TotalRents	Jane Doe	3	John Doe	2
Name	TotalRents						
Jane Doe	3						
John Doe	2						

❓ Debe usar: GROUP BY, COUNT(), ORDER BY, LIMIT

3. Consulta con múltiples JOINS anidados

❓ Genere un SELECT que devuelva lo siguiente:

- Nombre del cliente
- Nombre del libro
- Nombre del autor
- Estado del alquiler (Rents.State)

❓ Debe manejar el caso en que un libro no tenga autor

Para poder manejar el caso, se tuvo que agregar una línea más a la tabla Rents, para incluir el libro que no tiene autor,

Se seleccionó customer.name, book.name, y con COALESE se le ordenó que en la tabla autor verificara si el Name existía en la tabla Book, de encontrarse Null indicar que no había autor “No author”, el COALESE hace un barrido de izquierda a derecha verificando la información, y también Rents con su respectivo estado.

Se utilizaron 2 INNER JOIN tanto para Customer como para Books, esto para que brindará únicamente la información que coincide con la tabla Rents, y LEFT JOIN para Author para

que se nos brindara toda la información de Rent y toda la información de Atuhor, incluso con los resultados NULOS

```
SELECT Customers.Name AS Customer,
Books.Name AS Book,
COALESCE(Authors.Name, 'No Author') AS Author,
Rents.State AS Status
FROM Rents
INNER JOIN Customers ON Rents.Customer_ID = Customers.ID
INNER JOIN Books ON Rents.Book_ID = Books.ID
LEFT JOIN Authors ON Books.Author_ID = Authors.ID;
```

SQL ▾

< 1 / 1 > 1 - 6 of 6

SELECT Customers.Name AS Customer,
Books.Name AS Book,
COALESCE(Authors.Name, 'No Author') AS Author,
Rents.State AS Status
FROM Rents
INNER JOIN Customers ON Rents.Customer_ID = Customers.ID
INNER JOIN Books ON Rents.Book_ID = Books.ID
LEFT JOIN Authors ON Books.Author_ID = Authors.ID;

Customer	Book	Author	Status
Jane Doe	Don Quijote	Miguel de Cervantes	Returned
Jane Doe	La divina comedia	Dante Alighieri	Returned
John Doe	Don Quijote	Miguel de Cervantes	On time
John Doe	Vagabond1-3	Takehiko Inoue	On time
Jane Doe	La divina comedia	Dante Alighieri	Over due
Luke Skywalker	The Book of the 5 Rings	No Author	Over due